
AI Frontier: An Interactive Dashboard for Mapping the LLM Landscape

Avo Sarkissian

Electrical and Computer Engineering
Northeastern University
Boston, MA 02115
avohsarkissian@gmail.com

Abstract

The number of commercially available large language models has grown to the point where no single leaderboard covers all the dimensions that matter for practical use: price, throughput, latency, context window size, and benchmark performance. AI Frontier is an interactive dashboard that aggregates live data for every hosted language model Artificial Analysis benchmarks — 155 of them across 31 providers at the time of writing — refreshed every hour, and makes the full competitive landscape navigable through ten specialized views. Counts move hourly as models are added and delisted; the dashboard’s header stats are authoritative. The system is written entirely in Python and requires no JavaScript build step: the same chart code runs server-side under Plotly Dash for local development and client-side under Pyodide (WebAssembly) on the deployed static site. This report covers the data pipeline, composite intelligence scoring methodology, visualization type selection, and the design decisions that shaped the final tool.

1 Introduction

Selecting a large language model for a real application used to be straightforward — a handful of options existed and the best one for most tasks was obvious. That changed quickly. As of spring 2026, over 29 companies offer models through hosted APIs, each making different tradeoffs in price, capability, and latency. A model that scores 70 on reasoning benchmarks might cost twenty times more per token than one that scores 65. A local model that fits on a consumer GPU might outperform a paid API for a narrow task. Without a way to see all of this together, model selection is largely guesswork.

Existing leaderboards do one thing well. The LMSYS Chatbot Arena [2] ranks models on human preference. The Open LLM Leaderboard [3] tracks open-weight models on academic benchmarks. But neither shows live pricing, and most don’t update on a daily or hourly schedule. Provider dashboards show pricing but not quality scores. There is no single tool that combines current benchmark performance, price, throughput, and hardware requirements for both API-hosted and locally-runable models in one place.

AI Frontier fills that gap. The core idea is simple: pull current price and performance data for every major model on a regular schedule and surface it through enough visualization types that different kinds of users can answer different kinds of questions. A developer wants to know the cheapest model above a quality threshold. A researcher wants to see which provider is consistently at the top. A student wants to know which models will fit in their GPU’s memory. Each of those questions requires a different view of the same underlying data.

The project was built as part of EECE 5642 (Data Visualization) at Northeastern University. The dashboard is publicly accessible and the full source is open at <https://github.com/Avo-Sarkissian/AI-Frontier-Database>.

2 Data Sources and Pipeline

2.1 Cloud Model Data

The primary data source is the Artificial Analysis API [1], which tracks pricing and benchmark data for commercially hosted LLMs. For each model it provides: price per million tokens (input and output, at the cheapest available provider), median tokens per second (throughput), median time-to-first-token (latency), context window size, and a composite intelligence score. The API is queried once per hour by a background thread that starts when the application launches.

Each successful API response is written to a CSV cache and also saved as a timestamped snapshot in a history directory. Over 90 daily snapshots have accumulated. No tab currently reads them: a price-over-time view was planned and is not built, and the snapshots additionally span a change in the price basis on 2026-05-06, so any such chart must not cross that boundary without restating the earlier values.

A blended price per million tokens is computed from the raw input/output prices using a 3:1 output-to-input ratio, approximating real-world usage patterns. This weighting is ours, not Artificial Analysis's: AA publishes the opposite blend (3 parts input to 1 part output), so prices here read roughly 1.9x theirs. Output-weighting is the defensible basis for agentic and retrieval-augmented workloads, where generated tokens dominate spend. This collapses two numbers into one for display and sorting while keeping the full per-direction figures available in the detail table.

2.2 Local Model and Hardware Data

The second data source pairs a live scrape of open-weight models with a hand-maintained catalog of GPU hardware specifications. GPU memory bandwidth and VRAM capacity were pulled from official product pages (NVIDIA, AMD, Apple, Intel). Model parameter counts and quantization support were taken from HuggingFace model cards [10].

The GPU catalog is static — it gets updated when significant new hardware generations are released — while the open-weight model list is scraped hourly by `data/local_scraper.py` on the same cadence as the hosted-model data. It backs the Run Local tab, which estimates whether a given model fits in a selected GPU's memory and how fast it would run.

3 Intelligence Score and Benchmark Methodology

The intelligence score shown throughout the dashboard is inherited from Artificial Analysis, which aggregates results from multiple public evaluations into a single 0–100 composite. The score covers four categories:

- **Reasoning:** MMLU-Pro [4] and GPQA [5], testing factual knowledge and multi-step deductive reasoning.
- **Coding:** LiveCodeBench [6] and SciCode [7], evaluating writing and debugging real programs.
- **Math:** AIME and MATH-500 [8], covering competition-style quantitative problems.
- **Knowledge:** Humanity's Last Exam [9], a recently released benchmark of expert-level questions across scientific domains.

Using multiple benchmarks matters here. A model fine-tuned heavily on MMLU training data might rank first on that test alone but fall to the middle on LiveCodeBench. Averaging across categories rewards models that are consistently capable rather than narrowly optimized, and it lets API-hosted and local open-weight models be compared on the same scale.

The main limitation of composite scores is that they smooth over real differences. Two models with the same score can have completely different strength profiles — one excellent at coding and weak

at math, the other the reverse. The Compare tab (radar chart) exists specifically to surface these within-score differences.

4 Dashboard Design

4.1 Tab Structure

The dashboard has ten tabs, each targeting a specific question: **Overview** (cost vs. quality for every model), **Agent Stack** (three-tier model recommendations for agentic workflows), **Landscape** (treemap of the industry by provider), **Rankings** (top 25 by intelligence, value, or speed), **Compare** (radar chart for up to five models), **Budget** (projected monthly API cost from a token volume), **Table** (full sortable list of every tracked model), **Run Local** (VRAM compatibility and speed estimation), **Image Gen** (ELO-ranked image generation leaderboard), and **Video Gen** (ranked video generation comparison, from a curated dataset rather than a live scrape).

4.2 Visualization Choices

Overview: bubble scatter with Pareto frontier. The main challenge with comparing 255 models simultaneously is that a ranked list hides tradeoffs. The bubble scatter encodes three variables at once: x-axis is log-scaled price, y-axis is quality score, and bubble size represents throughput. Log scale on price is necessary because model prices span roughly three orders of magnitude — from under \$0.01 to over \$15 per million tokens — and a linear axis compresses most of the data into one corner of the chart.

A dotted Pareto frontier connects models that are undominated in the price-quality tradeoff: these are the points where any improvement in quality requires paying more, and any cost reduction means accepting lower quality. The frontier is computed in Pandas on each data load using a standard non-dominated sorting pass over price and quality. Visually, points on or near this curve represent the best practical choices for most real applications.

Each provider is assigned both a color and a distinct marker shape. The redundant encoding handles colorblind accessibility without requiring a separate mode or a legend that distinguishes only by hue.

Rankings: horizontal bars with tier separators. For one-dimensional ranked comparisons, horizontal bar charts are clearer than scatter plots — the comparison axis is explicit and label legibility is better horizontally. Tier separator lines are drawn wherever the score gap between adjacent models exceeds a fixed threshold, marking meaningful performance breaks rather than implying uniform spacing across the full range.

Compare: radar chart. Head-to-head comparison across five dimensions (intelligence, speed, affordability, context window, latency) is a natural use case for a radar chart. Each axis is normalized so all five dimensions are comparable on the same scale. The limitation of radar charts for this purpose is that interpretation depends on axis ordering, which is fixed in the current implementation. Reordering axes would change the visible shape of a model’s profile without changing any underlying values.

Landscape: treemap. A treemap where tile area encodes model count per provider and color intensity encodes average intelligence score gives a quick sense of industry structure — which providers have the most models, which cluster at the top, which are new entrants with a narrow catalog. The treemap is not suitable for precise value comparison, but it communicates structural patterns that scatter plots miss. A ranked bar chart directly below it allows precise comparison of the same providers.

Deferred: price-over-time line chart. The historical snapshot system was built to support this, and the market does move fast enough to justify it. It is not shipped: the snapshots span a change in the price basis on 2026-05-06, so a naive line would show a doubling that never happened. Publishing a misleading trend is worse than publishing none.

Run Local: VRAM compatibility calculator. This tab addresses a question that no existing public tool answers directly: which open-weight models can I actually run on my hardware? The user selects a GPU and a quantization level. The dashboard filters the catalog to models whose esti-

mated VRAM requirement fits the selected GPU’s memory and displays estimated inference speed in tokens per second.

VRAM requirements scale approximately with parameter count and bit-width. The formula accounts for model weights, key-value cache overhead, and activation memory. Quantization levels range from full precision (FP16/BF16) through 8-bit and 4-bit integer approximations, each roughly halving or quartering the memory footprint relative to the one above.

4.3 Design Principles

Three decisions carried through all ten tabs.

Data-ink ratio. Gridlines, outer frames, and chart borders were removed wherever they add visual weight without adding information [11]. The dark background with high-contrast data elements reduces perceived clutter compared to a white background with equivalent data density — the background recedes and the data elements stand out.

Colorblind accessibility. The nine largest providers each get both a color and a distinct marker shape — a redundant encoding that survives color-vision deficiency — kept consistent across all ten tabs. The remaining providers share a neutral “Other” series, because a categorical palette cannot separate thirty-plus categories reliably for any viewer. A user who learns the color scheme (or the shapes) can navigate between tabs without re-reading a legend each time.

Appropriate scales. Price axes use log scale throughout. Context window sizes, which range from 8K to over 1 million tokens, also use log scale. Linear scales for heavily right-skewed distributions compress the interesting variation at the low end, where most models actually sit.

4.4 Technical Stack

The dashboard runs on Plotly Dash, which compiles Python component trees into a live web interface without a JavaScript build step. All visualization logic is in Python using Plotly’s graph objects API; Dash handles HTTP routing, callback wiring, and component state.

Choosing a single language for the full stack — data pipeline, computation, and UI — simplified deployment. There is no Node.js dependency and no separate API server. The project deployed on Render [12] as a single Python process until 2026-06-23, and now ships as a static site on GitHub Pages: the same Python chart code is executed in the browser by Pyodide (WebAssembly), and an hourly GitHub Actions job performs the scrape that browsers cannot (the Artificial Analysis API sends no CORS headers).

The tradeoff moved with the architecture. Under Dash, interactions that a React application would handle locally needed a server round-trip. Under Pyodide there is no round-trip, but the browser downloads a 4 MB Python bundle before the controls become live, so the site paints from pre-rendered figures first and upgrades to interactive once that load completes.

5 Contributions

What AI Frontier adds relative to existing tools is the combination of live data with multi-modal coverage. Static leaderboards go stale within days of a new model release. Most pricing tools don’t include benchmark scores. AI Frontier scrapes both automatically, covers text, image, and video generation in a single dashboard, and includes local open-weight models alongside API services — a side-by-side comparison that does not exist elsewhere in a unified interface.

The Agent Stack tab is a practical extension of the core data: given a workflow type, it recommends a three-tier model setup (reasoning, balanced, and fast tiers) and surfaces specific model options at each tier. For users building agentic pipelines, this translates the raw leaderboard data into a concrete configuration decision.

The full dataset pipeline and all visualization modules are open-source. The dashboard has served live data continuously since March 2026.

6 Future Work

Several extensions are worth building. Per-category breakdown scores (reasoning, coding, math, knowledge separately) are collected by Artificial Analysis but not yet surfaced in the Compare tab — adding them would let users make more targeted selections than a single composite number allows.

The Budget tab currently estimates cost from a token count without accounting for prompt caching or multi-turn context accumulation. Both matter for production API usage and are straightforward to model given the existing pricing data.

Measured API latency by region is the other obvious addition. Published time-to-first-token medians from provider spec sheets are not always consistent with real-world performance, especially under load. The hourly polling infrastructure is already in place; extending it to include synthetic request timing would give a more complete picture of real operational cost.

References

References

- [1] Artificial Analysis. Artificial Analysis API: LLM performance and pricing benchmarks. <https://artificialanalysis.ai>, 2024.
- [2] W.-L. Chiang, L. Zheng, Y. Sheng, A. N. Angelopoulos, T. Li, D. Li, H. Zhang, B. Zhu, M. Jordan, J. E. Gonzalez, and I. Stoica. Chatbot arena: An open platform for evaluating LLMs by human preference. *arXiv preprint arXiv:2403.04132*, 2024.
- [3] Hugging Face. Open LLM Leaderboard. https://huggingface.co/spaces/HuggingFaceH4/open_llm_leaderboard, 2023.
- [4] Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang, T. Li, M. Ku, K. Wang, A. Zhuang, R. Fan, X. Yue, and W. Chen. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. *arXiv preprint arXiv:2406.01574*, 2024.
- [5] D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman. GPQA: A graduate-level google-proof Q&A benchmark. *arXiv preprint arXiv:2311.12022*, 2023.
- [6] N. Jain, K. Han, A. Gu, W. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.
- [7] M. Tian, Y. Fang, J. Liu, Y. Yu, Y. Li, K. McAlpine, P. Heinzelman, D. Lymperopoulos, J. Cheng, J. Smith, A. Shrivastava, Q. Xu, B. Zhou, and H. Ji. SciCode: A research coding benchmark curated by scientists. *arXiv preprint arXiv:2407.13168*, 2024.
- [8] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- [9] D. Phan, S. Pawar, D. Kambhampati, et al. Humanity’s Last Exam. Scale AI, 2025. <https://scale.com/hle>.
- [10] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of EMNLP: System Demonstrations*, pages 38–45, 2020.
- [11] E. R. Tufte. *The Visual Display of Quantitative Information*. Graphics Press, 2nd edition, 2001.
- [12] Render. Render cloud application platform. <https://render.com>, 2024.